O Futuro dos Desenvolvedores de Software com IA

https://gemini.google.com/app/42358fc438445266

User prompt: Eu escrevi uma espécie de brain storming nesse meu caderno em uma mistura de inglês e alemão. Você consegue extrair o texto escrito? Usei uma mistura de letra cursiva e letra bastão. Se algo não estiver claro na escrita, me pergunte.

Response: Aqui está a transcrição de todas as 6 páginas do seu caderno. Organizei página por página, mantendo os trechos rasurados indicados como `[rasurado]` e pequenas dúvidas de leitura indicadas com `[?]`.

Página 1 (Imagem: `20261018_192016.jpg`)

Why do I think AI won't replace software developers Wieso glaube ich, dass AI Programmierer nicht ersetzt wird Das liegt vor allem um Responsabilität und Balance of interests.

It is indeed wahr, dass wir können Software entwickeln at a extremely high speed, dass wir bis jetzt kaum uns kaum vorstellen könnten. Nichtsdestotrotz ist es schneller, den Code zu kriegen als den Code zu verstehen und alle seine Wirkungen zu verstehen.

Auch wenn ich den Code schon habe und ich ihn gar nicht geschrieben habe, werde ich dafür verantwortlich sein, für das gute und das schlechte für das was funktioniert und das, was nicht funktioniert.

Página 2 (Imagem: `20261018_192030.jpg`)

Wenn wir AI Werkzeuge verwenden sind wir Operatore für so ein Werkzeug, und sind dafür verantwortlich. Ohne Programmierer würden `[rasurado]` noch jemand gebraucht sein, um das AI Werkzeug zu steuern und dieses jemand würde der neue Operator sein. Wenn so jemand kein Entwickler ist, nennen wir sie "Vibe Coders", hauptsächlich weil sie nicht von Softwareentwicklung verstehen.

Trotzdem werden sie für das Software verantwortlich sein. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not.

Not having a programmer would look more or less like this from my perspective:

The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing `[rasurado]` programmers would mean they being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. `[rasurado]` You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately `[rasurado]` we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belong to whoever is selling you the AI tool.

I think the last scenario is the closest to where...

Página 3 (Imagem: `20261018_192040.jpg`)

...we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenues, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies.

When you hire a developer, you expect adaptability and tailored behavior from that developer.

In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior.

So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that?

I don't think so. I think we can do a lot if we create manners to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: "what we expect of you in the first 3 months" and some implicit, although many times defined by common sense, like:

- Is the code he writes readable or too confusing?
- Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs?

Página 4 (Imagem: `20261018_192053.jpg`)

- Can I trust that this coworker thought about possible edge cases?
- Are the PRs easy to review?

There is much which is very hard to actually measure, although quite important.

But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the following capabilities on your system in the past:

- Login
- Sign up
- Account Closure
- Account reopening

You might not care about each line of code, but you do care about behavior of your application. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether the implementation satisfies or not the `[rasurado]` integration tests you already had for that specific feature. The

goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system.

The more features your system has, the more tests you can create.

Página 5 (Imagem: `20261018_192102.jpg`)

Other things can be measured as well. You could implement fitness functions to help you create tests for things like:

- Performance
- Does this implementation respect the architecture we have for our system?

I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers ask questions to business people.

This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task.

Back to my previous accountability argument, could we now make `[rasurado]` agents accountable? No, we still can't. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent

These three roles can be played by the same person, but they don't have to, and ultimate responsibility lies on 2.

But a company can create a benchmark and hire an agent which performs better than any...

Página 6 (Imagem: `20261018_192110.jpg`)

...agent they were capable of creating. Honestly, only item 3 could be from outside the company I believe.

So yeah, I believe this is the closest we could have to agents replacing `[rasurado]` software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better.

One final thought: why would this be so hard? Because everyone `[rasurado]` wants a plug and play solution, and no one wants the overhead of creating benchmarks.

Caso queira confirmar algum trecho que rasurou ou tirar dúvidas sobre alguma frase específica, é só avisar!

User prompt: Você teve alguma dúvida durante a extração do texto? Algo que não estava claro?

Response: No geral, sua letra é bastante legível e o raciocínio é contínuo, o que ajudou muito a entender o contexto! Porém, houve alguns pequenos pontos em que a grafia ou o efeito de luz/sombra deixaram margem para dúvida.

Abaixo estão os únicos trechos onde fiquei ligeiramente em dúvida:

1. Página 1 (`20261018_192016.jpg`)

- **Trecho:** `Das liegt vor allem um Responsabilität und Balance of interests.`
    - **Dúvida:** Na escrita em alemão, no meio da frase parece que você misturou com inglês (`Responsabilität` em vez de *Verantwortung* ou *Responsibilität*). A leitura da palavra foi clara, mas vale confirmar se era exatamente essa a palavra que você pretendia usar.

2. Página 2 (`20261018_192030.jpg`)

- **Trecho:** `Replacing [rasurado] programmers would mean they being treated the same way as programmers are.`
    - **Dúvida:** Logo após a palavra rasurada, parece estar escrito `programmers` ou `non programmers`? Na leitura assumi `programmers`, mas como o trecho fala da substituição por não-programadores / gerentes, vale conferir o sentido original.

3. Página 3 (`20261018_192040.jpg`)

- **Trecho:** `I think we can do a lot if we create manners to evaluate the agents...`
    - **Dúvida:** A palavra parece `manners` ("modos/maneiras"), mas pelo contexto técnico poderia ser `measures` ("medidas") ou `mechanisms` ("mecanismos")?

4. Página 5 (`20261018_192102.jpg`)

- **Trecho:** `This is very similar to having code developers creating their own benchmarks...`
    - **Dúvida:** Antes de *developers*, a palavra parece `code developers` ou `core developers`?

Se você quiser ajustar ou confirmar algum desses trechos específicos, me avise! No mais, a transcrição seguiu exatamente o que estava legível na página.

User prompt: Me entregue todo o texto extraído, traduzindo o que era alemão para o inglês. Acredito que a sua dúvida de core developers era simplesmente code developers, ou programmers. Além disso, as páginas estão na ordem certa ou invertida? Se estiverem na ordem invertida, coloque o texto na ordem correta.

Response: As imagens do seu caderno foram tiradas em **ordem inversa** (a foto `20261018_192016.jpg` é a Página 1 e a foto `20261018_192110.jpg` é a Página 6 na sequência do seu raciocínio).

Abaixo está o **texto completo e na ordem cronológica correta (da Página 1 até a Página 6)**, já com as partes em alemão traduzidas para o inglês (mantendo o estilo misto do seu *brainstorming*) e com o ajuste de `code developers` confirmado por você:

Page 1 (Image: `20261018_192016.jpg`)

Why do I think AI won't replace software developers Why do I believe that AI won't replace programmers That is mainly about responsibility and balance of interests.

It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects.

Even if I already have the code and I didn't write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn't work.

Page 2 (Image: `20261018_192030.jpg`)

When we use AI tools we are operators for such a tool, and are responsible for it. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If such someone is not a developer, we call them "Vibe Coders", mainly because they don't understand software development.

Nevertheless, they will be responsible for the software. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not.

Not having a programmer would look more or less like this from my perspective:

The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers would mean they being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belong to whoever is selling you the AI tool.

I think the last scenario is the closest to where...

Page 3 (Image: `20261018_192040.jpg`)

...we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenues, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies.

When you hire a developer, you expect adaptability and tailored behavior from that developer.

In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior.

So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that?

I don't think so. I think we can do a lot if we create manners to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: "what we expect of you in the first 3 months" and some implicit, although many times defined by common sense, like:

- Is the code he writes readable or too confusing?
- Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs?

Page 4 (Image: `20261018_192053.jpg`)

- Can I trust that this coworker thought about possible edge cases?
- Are the PRs easy to review?

There is much which is very hard to actually measure, although quite important.

But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the following capabilities on your system in the past:

- Login
- Sign up
- Account Closure
- Account reopening

You might not care about each line of code, but you do care about behavior of your application. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether the implementation satisfies or not the integration tests you already had for that specific feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system.

The more features your system has, the more tests you can create.

Page 5 (Image: `20261018_192102.jpg`)

Other things can be measured as well. You could implement fitness functions to help you create tests for things like:

- Performance
- Does this implementation respect the architecture we have for our system?

I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers ask questions to business people.

This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task.

Back to my previous accountability argument, could we now make agents accountable? No, we still can't. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent

These three roles can be played by the same person, but they don't have to, and ultimate responsibility lies on 2.

But a company can create a benchmark and hire an agent which performs better than any...

Page 6 (Image: `20261018_192110.jpg`)

...agent they were capable of creating. Honestly, only item 3 could be from outside the company I believe.

So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better.

One final thought: why would this be so hard? Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks.

User prompt: I have this now: How far I believe we can go in AI replacing software developers I was originally trying to write an article about “why I think AI will not replace software developers”, however, I came to find gaps in my own logical reasoning. That said, I still don’t believe in full removal but, as many have already noticed, a large re-estructuring of the workflow of developers, but not much has changed regarding what is expected them to deliver. Below are my thoughts, hope you like it. So, most of my thinking goes mainly around responsibility and balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects. Even if I already have the code and I didn’t write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn’t work. The new bottleneck has become understanding the code and building trust around it, not writing it anymore. When we use AI tools we are operators for such a tool, and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If such someone is not a developer and is responsible for operating an AI tool, but with no knowledge of programming, we call them “Vibe Coders”, mainly because they treat the code as a “black box”. Nevertheless, they will be responsible for the software. The point is,

as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not. Not having a programmer would look more or less like this from my perspective: The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers with an agent would mean the agent being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belong to whoever is selling you the AI tool. I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenue, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies. When you hire a developer, you expect adaptability and tailored behavior from that developer. In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior. So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that? I don't think so. I think we can do a lot if we create manners to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: "what we expect of you in the first 3 months" and some implicit, although many times defined by common sense, like: - Is the code he writes readable or too confusing? - Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs? - Can I trust that this coworker thought about possible edge cases? - Are the PRs easy to review? There is much which is very hard to actually measure, although quite important. But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the following capabilities on your system in the past: - Login - Sign up - Account Closure - Account reopening You might not care about each line of code, but you do care about behavior of your application and scalability. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether the implementation satisfies or not the integration tests you already had for that specific feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system. The more features your system has, the more tests you can create. Other things can be measured as well. You could implement fitness functions to help you create tests for things like: - Performance - Does this implementation respect the architecture we have for our system? I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers ask questions to business people. This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task. Back to my previous accountability argument, could we now make agents accountable? No, we still can't. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent These three roles can be played by the same person, but they don't have to, and ultimate responsibility lies on 2. But a company can create a benchmark and hire an agent which performs better than any agent they were capable of creating. You could have something like someone selling you an

specialized agent for fintechs which performs at a specific level on, let's say, fintech benchmarks. Honestly, only item 3 could be from outside the company I believe. So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications and features that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better. A few final thoughts: Why would this be so hard? 1— Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks. 2—It is very hard to test something like 'good design' and 'good architecture'. At least I don't know a clear, deterministic and mathematical away do evaluate whether a code follows the SOLID principles, or if fullfils precepts from clean architecture. These are my thoughts, hope you like it :) Are there clear grammatical mistakes in this article?

Response: Seu artigo está muito bem articulado e argumentado. Existem, no entanto, algumas correções gramaticais e pequenas melhorias de fluência em inglês para torná-lo 100% natural.

Abaixo estão os **erros gramaticais e correções recomendadas**, agrupados por categoria:

1. Erros Gramaticais Diretos

- **Na Introdução:**
  - **Original:** *"...re-estructuring of the workflow of developers, but not much has changed regarding what is expected them to deliver."*
  - **Correção:** *"...restructuring of the workflow of developers, but not much has changed regarding what is expected **of** them to deliver."* (Faltava a preposição *of*).
  - **Original:** *"...hope you like it."*
  - **Correção:** *"...hope you **enjoy** it"* ou *"...hope you like **them**"* (já que se refere a *thoughts*, no plural).
- **No parágrafo do "Vibe Coders":**
  - **Original:** *"If such someone is not a developer..."*
  - **Correção:** *"If **someone like that** is not a developer..."* ou *"If **such a person** is not a developer..."* (*Such someone* não é gramaticalmente correto em inglês).
- **No parágrafo da "Responsabilidade":**
  - **Original:** *"Or you could say that responsibility belong to..."*
  - **Correção:** *"Or you could say that responsibility **belongs** to..."* (Concordância no singular).
- **Na seção das Perguntas/Avaliações:**
  - **Original:** *"Is the code he writes readable or too confusing?"*
  - **Correção:** *"Is the code **they write** readable or too confusing?"* (Em artigos modernos de tecnologia, usa-se o pronome neutro *they* para se referir a um dev genérico).

- **Na seção dos Testes/Benchmarking:**
    - **Original:** *"...check whether the implementation satisfies or not the integration tests..."*
    - **Correção:** *"...check whether **or not** the implementation satisfies the integration tests..."* (A ordem das palavras fica mais natural).
    - **Original:** *"...the same way we allow developers ask questions to business people."*
    - **Correção:** *"...the same way we allow developers **to** ask questions to business people."* (Faltava o *to* no infinitivo).
- **Nas Considerações Finais:**
    - **Original:** *"At least I don't know a clear, deterministic and mathematical away do evaluate whether a code follows..."*
    - **Correção:** *"At least I don't know a clear, deterministic, and mathematical **way to** evaluate whether **the** code follows..."* (*away do* foi um erro de digitação por *way to*).
    - **Original:** *"...or if fullfils precepts from clean architecture."*
    - **Correção:** *"...or if **it fulfills** precepts **of** clean architecture."* (Faltava o sujeito *it*, além da grafia correta de *fulfills* e a preposição *of*).

2. Ajustes de Fluência e Estilo (Recomendados)

- **Título:** *"How far I believe we can go in AI replacing software developers"*
    - **Sugestão mais natural:** *"How Far I Believe We Can Go with AI Replacing Software Developers"*
- **Uso da palavra *manners*:**
    - **Original:** *"...if we create manners to evaluate the agents..."*
    - **Sugestão:** *"Manners"* refere-se a boas maneiras/modos sociais. Para soluções técnicas, o ideal é usar **"mechanisms"**, **"ways"** ou **"frameworks"**.

Texto Corrigido na Íntegra (Pronto para Publicar)

How Far I Believe We Can Go with AI Replacing Software Developers

I was originally trying to write an article about "why I think AI will not replace software developers"; however, I came to find gaps in my own logical reasoning. That said, I still don't believe in full replacement, but rather—as many have already noticed—a large restructuring of the workflow of developers, with little changing regarding what is expected of them to deliver. Below are my thoughts; I hope you enjoy them.

Most of my thinking revolves around responsibility and the balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to generate code than to understand it and all of its side effects. Even if I already have the code and didn't write a single line of

it, I will still be responsible for it—for the good and the bad, for what works and what doesn't. The new bottleneck has become understanding the code and building trust around it, not writing it anymore.

When we use AI tools, we act as operators for those tools and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool, and that person would become the new operator. If such a person is not a developer and is responsible for operating an AI tool without any programming knowledge, we call them "Vibe Coders," mainly because they treat code as a "black box."

Nevertheless, they will still be held responsible for the software. The point is, as a software developer, I am much more conscious of all the consequences of my code—whether it scales, whether it is readable, and how it behaves.

Not having a programmer would look more or less like this from my perspective: The same person who is currently a non-technical manager would describe whatever implementation they want. From this perspective, you could say the manager is an operator, but it would be no different than a manager operating their team. Replacing programmers with an agent would mean the agent is treated the same way developers are. But who would be held accountable then?

I think this is where things start getting complicated. You could try to say we should hold AI accountable, but everyone knows it doesn't work like that. Ultimately, we know that whoever operates the AI is held accountable. Accountability could belong to the manager, but then they would simply be classified as a vibe coder. Or you could say that responsibility belongs to whoever sells you the AI tool.

I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI, and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, such as loss of revenue, so they are implicitly accountable. The difference is that the companies hiring the AI tools have much less power when trying to steer the behavior of these providers.

When you hire a developer, you expect adaptability and tailored behavior from that individual. In the case of hiring a company, you would technically get that, but it would be much harder for you to measure or correct unintended behavior.

So, is all lost? Are we stuck with the current organizational structure, meaning AI will change nothing about it?

I don't think so. I think we can achieve a lot if we create mechanisms to evaluate agents, allowing us to predict and measure standard behavior. Let me be a bit clearer on that. Today, we evaluate software developers first in job interviews, where we measure general technical knowledge. Once inside a company, developers go through a series of everyday evaluations—some explicit, like "what we expect of you in the first 3 months," and some implicit, defined by common sense, like:

- Is the code they write readable or too confusing?
- Is this coworker easy to manage? Or do I spend hours trying to explain tasks and reviewing buggy PRs?

- Can I trust that this coworker thought about possible edge cases?
- Are the PRs easy to review?

Much of this is very hard to actually measure, despite being crucial.

But regarding what we *can* measure, how do we proceed? We could define a baseline agent performance. Let’s imagine you implemented the following capabilities in your system in the past:

- Login
- Sign up
- Account Closure
- Account Reopening

You might not care about every single line of code, but you do care about the behavior of your application and its scalability. You could, for instance, take a snapshot of your application before these features were implemented, ask the agent to implement them, and check whether or not the implementation satisfies the integration tests you already had for those features. The goal here is to measure how well the agent can implement features in a way that is acceptable to your system.

The more features your system has, the more tests you can create.

Other aspects can be measured as well. You could implement fitness functions to help create tests for things like:

- Performance
- Whether the implementation respects the system's architecture

I believe we should allow and expect agents to ask us questions to produce better implementations, just as we allow developers to ask questions to business stakeholders.

This is very similar to software developers creating their own benchmarks to evaluate agents, developers, or teams. Agents can be improved and regularly evaluated against these benchmarks. They can be modified and re-evaluated; we can have different benchmarks for different roles (e.g., Frontend, Backend, SRE), and we can even evaluate cost per task.

Back to my previous accountability argument: can we now make agents accountable? No, we still can’t. The accountable parties would be:

1. Whoever creates the benchmarks
2. Whoever approves the code to go to production
3. Whoever creates the agent

These three roles can be played by the same person, but they don’t have to be, and ultimate responsibility lies with #2.

However, a company could create a benchmark and hire an agent that performs better than any agent they were capable of creating internally. You could have someone selling a specialized agent for fintechs that performs at a specific level on fintech-specific benchmarks. Honestly, I believe only item #3 could come from outside the company.

So yes, I believe this is the closest we could get to agents replacing software developers: a team structure in which standardized tests are applied to agents to evaluate how well they create specifications and features that satisfy expectations, where a company could even hire external agents simply because they perform better.

A few final thoughts: Why would this be so hard?

1. Everyone wants a plug-and-play solution, and no one wants the overhead of creating benchmarks.

2. It is very hard to test subjective concepts like "good design" and "good architecture." At least, I don't know a clear, deterministic, and mathematical way to evaluate whether code follows SOLID principles or fulfills the precepts of Clean Architecture.

These are my thoughts; I hope you enjoy them! :)

User prompt: E agora: How far I believe We Can Go With AI Replacing Software Developers I was originally trying to write an article about “why I think AI will not replace software developers”, however, I came to find gaps in my own logical reasoning. That said, I still don’t believe in full removal but, as many have already noticed, a large restructuring of the workflow of developers, but not much has changed regarding what is expected them to deliver. Below are my thoughts, hope you like them. So, most of my thinking goes mainly around responsibility and balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects. Even if I already have the code and I didn’t write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn’t work. The new bottleneck has become understanding the code and building trust around it, not writing it anymore. When we use AI tools we are operators for such a tool, and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If someone like that is not a developer and is responsible for operating an AI tool, but with no knowledge of programming, we call them “Vibe Coders”, mainly because they treat the code as a “black box”. Nevertheless, they will be responsible for the software. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not. Not having a programmer would look more or less like this from my perspective: The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers with an agent would mean the agent being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it

does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belongs to whoever is selling you the AI tool. I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenue, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies. When you hire a developer, you expect adaptability and tailored behavior from that developer. In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior. So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that? I don't think so. I think we can do a lot if we create mechanisms to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: "what we expect of you in the first 3 months" and some implicit, although many times defined by common sense, like: - Is the code they write readable or too confusing? - Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs? - Can I trust that this coworker thought about possible edge cases? - Are the PRs easy to review? There is much which is very hard to actually measure, although quite important. But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the following capabilities on your system in the past: - Login - Sign up - Account Closure - Account reopening You might not care about each line of code, but you do care about behavior of your application and scalability. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether or not the implementation satisfies or not the integration tests you already had for that specific feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system. The more features your system has, the more tests you can create. Other things can be measured as well. You could implement fitness functions to help you create tests for things like: - Performance - Does this implementation respect the architecture we have for our system? I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers to ask questions to business people. This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task. Back to my previous accountability argument, could we now make agents accountable? No, we still can't. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent These three roles can be played by the same person, but they don't have to, and ultimate responsibility lies on 2. But a company can create a benchmark and hire an agent which performs better than any agent they were capable of creating. You could have something like someone selling you an specialized agent for fintechs which performs at a specific level on, let's say, fintech benchmarks. Honestly, only item 3 could be from outside the company I believe. So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications and features that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better. A few final thoughts: Why would this be so hard? 1— Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks. 2—It is very hard to test something like 'good design' and

‘good architecture’. At least I don’t know a clear, deterministic and mathematical away to evaluate whether a code follows the SOLID principles, or if fullfils precepts from clean architecture. These are my thoughts, hope you like it :) Foram corrigidos os pontos?

Response: Você corrigiu quase **todos os pontos principais**, e a leitura agora está incomparavelmente melhor e mais fluida!

Ainda restaram **apenas 5 pequenos detalhes de digitação/redação** que passaram despercebidos. São ajustes simples de fazer:

Pequenos detalhes restantes a corrigir:

1. **Na Introdução:**
    - **Seu texto:** *“...regarding what is expected **them** to deliver.”*
    - **Correção:** *“...regarding what is expected **of them** to deliver.”* (Ainda falta a preposição **of**).
    - **Seu texto:** *“...hope you like **them**.”* / **No final:** *“...hope you like **it** :)”*
    - **Sugestão:** Padronize ambos. Se for "thoughts", use **them** (*"hope you like them"*).
2. **No meio do texto (Repetição de *or not*):**
    - **Seu texto:** *“...check **whether or not** the implementation satisfies **or not** the integration tests...”*
    - **Correção:** *“...check **whether or not** the implementation satisfies the integration tests...”* (Remover o segundo *or not*).
3. **Nas Considerações Finais (#2):**
    - **Seu texto:** *“...mathematical **away** to evaluate whether **a code** follows...”*
    - **Correção:** *“...mathematical **way** to evaluate whether **the code** (ou **code**) follows...”* (*away* ainda é um typo de *way*).
    - **Seu texto:** *“...or if **fullfils** precepts from clean architecture.”*
    - **Correção:** *“...or if **it fulfills** precepts **of** clean architecture.”* (Faltou o sujeito **it**, a grafia correta com um 'l' **fulfills**, e a preposição **of**).

Sua versão final 100% polida:

> **How Far I Believe We Can Go With AI Replacing Software Developers**
>
> I was originally trying to write an article about “why I think AI will not replace software developers”, however, I came to find gaps in my own logical reasoning. That said, I still don’t believe in full removal but, as many have already noticed, a large restructuring of the workflow of developers, but not much has changed regarding what is expected **of them** to deliver. Below are my thoughts, hope you like **them**.

So, most of my thinking goes mainly around responsibility and balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects. Even if I already have the code and I didn't write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn't work. The new bottleneck has become understanding the code and building trust around it, not writing it anymore. When we use AI tools we are operators for such a tool, and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If someone like that is not a developer and is responsible for operating an AI tool, but with no knowledge of programming, we call them "Vibe Coders", mainly because they treat the code as a "black box". Nevertheless, they will be responsible for the software. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not. Not having a programmer would look more or less like this from my perspective: The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers with an agent would mean the agent being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belongs to whoever is selling you the AI tool. I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenue, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies. When you hire a developer, you expect adaptability and tailored behavior from that developer. In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior. So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that? I don't think so. I think we can do a lot if we create mechanisms to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: "what we expect of you in the first 3 months" and some implicit, although many times defined by common sense, like:

- Is the code they write readable or too confusing?
- Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs?
- Can I trust that this coworker thought about possible edge cases?
- Are the PRs easy to review? There is much which is very hard to actually measure, although quite important. But on that which we can measure how can we proceed? We could look or define something like a baseline agent

performance. Lets imagine you implemented the following capabilities on your system in the past:

- Login
- Sign up
- Account Closure
- Account reopening You might not care about each line of code, but you do care about behavior of your application and scalability. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check **whether or not** the implementation satisfies the integration tests you already had for that specific feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system. The more features your system has, the more tests you can create. Other things can be measured as well. You could implement fitness functions to help you create tests for things like:
- Performance
- Does this implementation respect the architecture we have for our system? I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers to ask questions to business people. This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task. Back to my previous accountability argument, could we now make agents accountable? No, we still can’t. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent These three roles can be played by the same person, but they don’t have to, and ultimate responsibility lies on 2. But a company can create a benchmark and hire an agent which performs better than any agent they were capable of creating. You could have something like someone selling you an specialized agent for fintechs which performs at a specific level on, let’s say, fintech benchmarks. Honestly, only item 3 could be from outside the company I believe. So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications and features that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better. A few final thoughts: Why would this be so hard? 1 — Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks. 2 — It is very hard to test something like ‘good design’ and ‘good architecture’. At least I don’t know a clear, deterministic and mathematical **way** to evaluate whether code follows the SOLID principles, or if **it fulfills** precepts **of** clean architecture. These are my thoughts, hope you like **them** :)

User prompt: E agora: How far I believe We Can Go With AI Replacing Software Developers I was originally trying to write an article about “why I think AI will not replace software developers”, however, I came to find gaps in my own logical reasoning. That said, I still don’t believe in full removal but, as many have already noticed, a large restructuring of the workflow of developers, but not much has changed regarding what is expected of them to deliver. Below are my thoughts, hope you like them. So, most of my thinking goes mainly around responsibility and balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects. Even if I already have the code and I didn’t write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn’t work. The new bottleneck has become understanding the code and building trust around it, not writing it anymore. When we use AI tools we are operators for such a tool, and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If someone like that is not a developer and is responsible for operating an AI tool, but with no knowledge of programming, we call them “Vibe Coders”, mainly because they treat the code as a “black box”. Nevertheless, they will be responsible for the software. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not. Not having a programmer would look more or less like this from my perspective: The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers with an agent would mean the agent being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belongs to whoever is selling you the AI tool. I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenue, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies. When you hire a developer, you expect adaptability and tailored behavior from that developer. In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior. So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that? I don’t think so. I think we can do a lot if we create mechanisms to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: “what we expect of you in the first 3 months” and some implicit, although many times defined by common sense, like: - Is the code they write readable or too confusing? - Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs? - Can I trust that this coworker thought about possible edge cases? - Are the PRs easy to review? There is much which is very hard to actually measure, although quite important. But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the following capabilities on your system in the past: - Login - Sign up - Account Closure - Account reopening You might not care about each line of code, but you do care about behavior of your application and scalability. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether or not the implementation satisfies the integration tests you already had for that specific

feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system. The more features your system has, the more tests you can create. Other things can be measured as well. You could implement fitness functions to help you create tests for things like: - Performance - Does this implementation respect the architecture we have for our system? I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers to ask questions to business people. This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task. Back to my previous accountability argument, could we now make agents accountable? No, we still can't. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent These three roles can be played by the same person, but they don't have to, and ultimate responsibility lies on 2. But a company can create a benchmark and hire an agent which performs better than any agent they were capable of creating. You could have something like someone selling you an specialized agent for fintechs which performs at a specific level on, let's say, fintech benchmarks. Honestly, only item 3 could be from outside the company I believe. So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications and features that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better. A few final thoughts: Why would this be so hard? 1— Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks. 2—It is very hard to test something like 'good design' and 'good architecture'. At least I don't know a clear, deterministic and mathematical way to evaluate whether a code follows the SOLID principles, or if fulfills precepts of clean architecture. These are my thoughts, hope you like them :) Corretamente polida?

Response: Sim, o seu texto agora está **praticamente perfeito** e pronto para publicação! Todas as correções críticas de gramática, preposições e concordância que apontamos anteriormente foram resolvidas com sucesso.

Para que ele fique **100% impecável**, identificamos apenas **dois detalhes mínimos de digitação** que ainda sobraram nas últimas duas linhas do texto:

1. **Artigo antes da palavra "code" (Ponto 2 do final):**
    - **Seu texto:** *"...whether **a** code follows the SOLID principles..."*
    - **Ajuste:** Em inglês, *code* costuma ser não-contável nesse contexto. O ideal é remover o artigo **a** (*"whether code follows..."*) ou usar **the code** (*"whether the code follows..."*).
2. **Sujeito no verbo "fulfills" (Ponto 2 do final):**
    - **Seu texto:** *"...or if **fulfills** precepts of clean architecture."*
    - **Ajuste:** Faltou o pronome/sujeito **it** antes do verbo (*"...or if **it** fulfills precepts of clean architecture."*).

Versão Final Totalmente Polida

How Far I Believe We Can Go With AI Replacing Software Developers

I was originally trying to write an article about “why I think AI will not replace software developers”, however, I came to find gaps in my own logical reasoning. That said, I still don’t believe in full removal but, as many have already noticed, a large restructuring of the workflow of developers, but not much has changed regarding what is expected of them to deliver. Below are my thoughts, hope you like them.

So, most of my thinking goes mainly around responsibility and balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects. Even if I already have the code and I didn’t write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn’t work. The new bottleneck has become understanding the code and building trust around it, not writing it anymore. When we use AI tools we are operators for such a tool, and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If someone like that is not a developer and is responsible for operating an AI tool, but with no knowledge of programming, we call them “Vibe Coders”, mainly because they treat the code as a “black box”. Nevertheless, they will be responsible for the software. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not. Not having a programmer would look more or less like this from my perspective: The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers with an agent would mean the agent being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belongs to whoever is selling you the AI tool. I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenue, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies. When you hire a developer, you expect adaptability and tailored behavior from that developer. In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior. So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that? I don’t think so. I think we can do a lot if we create mechanisms to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: “what we expect of you in the first 3 months” and some implicit, although many times defined by common sense, like:

- Is the code they write readable or too confusing?
- Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs?

- Can I trust that this coworker thought about possible edge cases?
- Are the PRs easy to review? There is much which is very hard to actually measure, although quite important. But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the following capabilities on your system in the past:
- Login
- Sign up
- Account Closure
- Account reopening You might not care about each line of code, but you do care about behavior of your application and scalability. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether or not the implementation satisfies the integration tests you already had for that specific feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system. The more features your system has, the more tests you can create. Other things can be measured as well. You could implement fitness functions to help you create tests for things like:
- Performance
- Does this implementation respect the architecture we have for our system? I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers to ask questions to business people. This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task. Back to my previous accountability argument, could we now make agents accountable? No, we still can’t. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent These three roles can be played by the same person, but they don’t have to, and ultimate responsibility lies on 2. But a company can create a benchmark and hire an agent which performs better than any agent they were capable of creating. You could have something like someone selling you an specialized agent for fintechs which performs at a specific level on, let’s say, fintech benchmarks. Honestly, only item 3 could be from outside the company I believe. So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications and features that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better. A few final thoughts: Why would this be so hard? 1 — Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks. 2 — It is very hard to test something like ‘good design’ and ‘good architecture’. At least I don’t know a clear, deterministic and mathematical way to evaluate whether code

follows the SOLID principles, or if **it** fulfills precepts of clean architecture. These are my thoughts, hope you like them :)

User prompt: E agora: How far I believe We Can Go With AI Replacing Software Developers I was originally trying to write an article about “why I think AI will not replace software developers”, however, I came to find gaps in my own logical reasoning. That said, I still don’t believe in full removal but, as many have already noticed, a large restructuring of the workflow of developers, but not much has changed regarding what is expected of them to deliver. Below are my thoughts, hope you like them. So, most of my thinking goes mainly around responsibility and balance of interests. It is indeed true that we can develop software at an extremely high speed that until now we could barely imagine. Nevertheless, it is faster to get the code than to understand the code and understand all of its effects. Even if I already have the code and I didn’t write it at all, I will be responsible for it, for the good and the bad, for what works and what doesn’t work. The new bottleneck has become understanding the code and building trust around it, not writing it anymore. When we use AI tools we are operators for such a tool, and are responsible for the outcome. Without programmers, someone else would still be needed to control the AI tool and this someone would be the new operator. If someone like that is not a developer and is responsible for operating an AI tool, but with no knowledge of programming, we call them “Vibe Coders”, mainly because they treat the code as a “black box”. Nevertheless, they will be responsible for the software. The point is, as a software developer I am much more conscious of all of the consequences of my code, whether it scales or not, if it is readable or not. Not having a programmer would look more or less like this from my perspective: The same person who is now something like a non technical manager would describe whatever implementation it wants. In this perspective you could say the manager is an Operator, but it would be no less than a manager is actually operating its team. Replacing programmers with an agent would mean the agent being treated the same way as programmers are. But who would be held accountable then? I think this is where things start getting complicated. You could try to say we would hold AI accountable, but everyone knows it does not work like this. Ultimately we know that whoever is operating AI is held accountable. Accountability could belong to the manager, but then he would have just been classified as a vibe coder. Or you could say that responsibility belongs to whoever is selling you the AI tool. I think the last scenario is the closest to where we are right now, or the closest we could get. Companies like Anthropic, OpenAI and Google are explicit about not being accountable, but you could interpret things another way. Poor service from these companies does have consequences, like loss of revenue, so they are actually accountable, the difference is that the companies which hire the AI tools have a much smaller power when trying to steer behavior of these companies. When you hire a developer, you expect adaptability and tailored behavior from that developer. In the case of you hiring a company, you would actually have that, but it would be much harder for you to measure this, and for you to replace unintended behavior. So, is it all lost? Are we stuck with the current organizational structure and having AI will absolutely change nothing about that? I don’t think so. I think we can do a lot if we create mechanisms to evaluate the agents which will allow us to try to predict and measure standard behavior. Let me be a bit clearer on that. Today we evaluate software developers first on job interviews in which we measure first general knowledge about tech, and that is it. Once inside a company, developers go through a series of everyday evaluations, some explicit, like: “what we expect of you in the first 3 months” and some implicit, although many times defined by common sense, like: - Is the code they write readable or too confusing? - Is this coworker easy to manage? Or do I spend hours and more hours trying to explain tasks and reviewing buggy PRs? - Can I trust that this coworker thought about possible edge cases? - Are the PRs easy to review? There is much which is very hard to actually measure, although quite important. But on that which we can measure how can we proceed? We could look or define something like a baseline agent performance. Lets imagine you implemented the

following capabilities on your system in the past: - Login - Sign up - Account Closure - Account reopening You might not care about each line of code, but you do care about behavior of your application and scalability. You could, for instance, take a snapshot of your application before the implementation of these features, then ask the agent to implement these features and check whether or not the implementation satisfies the integration tests you already had for that specific feature. The goal here is to be able to measure how well the agent can implement features in a way that is considered acceptable by your system. The more features your system has, the more tests you can create. Other things can be measured as well. You could implement fitness functions to help you create tests for things like: - Performance - Does this implementation respect the architecture we have for our system? I believe we should allow and expect such agents to ask us questions in order to create better implementations, the same way we allow developers to ask questions to business people. This is very similar to having code developers creating their own benchmarks to evaluate agents, developers or teams actually. Agents can be improved and regularly evaluated against such benchmarks. Agents can be modified and re-evaluated, we can have different benchmarks for different roles i.e. Front end and back end and SRE, and we can even evaluate cost per task. Back to my previous accountability argument, could we now make agents accountable? No, we still can't. The accountable ones would be: 1- Whoever creates the benchmarks 2- Whoever approves the code to go to prod 3- Whoever creates the agent These three roles can be played by the same person, but they don't have to, and ultimate responsibility lies on 2. But a company can create a benchmark and hire an agent which performs better than any agent they were capable of creating. You could have something like someone selling you an specialized agent for fintechs which performs at a specific level on, let's say, fintech benchmarks. Honestly, only item 3 could be from outside the company I believe. So yeah, I believe this is the closest we could have to agents replacing software developers. i.e., A team structure in which standardized tests are applied to the agents to properly evaluate how well they can create specifications and features that satisfy what we expect and in which a company could even hire external agents from other companies just because they are better. A few final thoughts: Why would this be so hard? 1— Because everyone wants a plug and play solution, and no one wants the overhead of creating benchmarks. 2—It is very hard to test something like 'good design' and 'good architecture'. At least I don't know a clear, deterministic and mathematical way to evaluate whether code follows the SOLID principles, or if it fulfills precepts of clean architecture. These are my thoughts, hope you like them :) Está correto?

Response: **Perfeito! O texto está 100% correto, natural e gramaticalmente polido.**

Todas as pequenas imperfeições de digitação, pontuação, artigos e preposições foram totalmente resolvidas. Pode publicar sem preocupações!

User prompt: Ok, agora me entregue uma imagem que represente esse código.

Response: Como não tenho a capacidade de gerar ou desenhar imagens diretamente, aqui estão **três sugestões de prompts detalhados** que você pode copiar e colar em geradores de imagem por IA (como Midjourney, DALL-E 3 ou Leonardo.ai) para criar uma imagem perfeita para o seu artigo:

Opção 1: Estilo Conceitual & Futurista (Ideal para capas do Medium/Substack)

> **Prompt:** A conceptual digital illustration about AI software engineering. In the foreground, a human software developer sits at a clean desk, reviewing floating holographic architectural diagrams and code benchmarks. Opposite to the human, a

> sleek semi-transparent humanoid AI agent works in parallel, connected by glowing nodes of data and unit test functions. Neon blue and dark gray color palette, highly detailed, minimalist tech art style, 8k resolution, cinematic lighting.

Opção 2: Metáfora de Avaliação & Benchmarks (Foco no tema do seu artigo)

> **Prompt:** A visual metaphor of evaluating AI coding agents. A large glowing digital leaderboard hovering in a dark futuristic server room, showing standardized benchmark tests, fitness functions, and performance metrics. Human engineers in the background inspecting and approving code pull requests on modern displays. Clean line art combined with glowing cybernetic elements, dark mode theme, corporate tech aesthetic.

Opção 3: Estilo Ilustração Editorial / Cartoon Moderno (Excelente para LinkedIn)

> **Prompt:** A modern flat vector illustration for a tech blog. On one side, a "Vibe Coder" looking at a glowing black box labeled "AI". On the other side, an experienced developer creating structured benchmark frameworks and automated test suites to control AI agents. Clean design, soft pastel tech colors, professional tech editorial illustration style.

Se preferir, você também pode usar ferramentas como o **DALL-E 3 (no ChatGPT Plus / Copilot)** ou **Midjourney** usando qualquer um dos prompts acima para gerar a imagem exata que você precisa!

User prompt: Você não consegue criar tal imagem?Se sim, por favor crie

Response: Como modelo de linguagem baseado em texto, não possuo a capacidade de gerar ou renderizar arquivos de imagem diretamente no chat.

No entanto, se você utilizar ferramentas de geração de imagem como o **DALL-E 3** (no ChatGPT/Copilot), **Midjourney**, **Adobe Firefly** ou **Leonardo.ai**, basta copiar e colar um dos prompts abaixo para obter o resultado desejado:

Prompt Recomendado (Para DALL-E / Midjourney):

> **Prompt:** A professional tech editorial illustration representing AI and software engineering accountability. A modern developer at a clean desk, reviewing floating holographic architectural diagrams, benchmark tests, and code quality metrics. Across the desk, a subtle glowing humanoid AI agent operates in sync, surrounded by floating pull-request checklists and fitness functions. Dark mode aesthetic, deep blue and slate gray palette, sleek vector style, highly detailed.

Se precisar de ajuda para ajustar o prompt ou preferir uma ideia em outro estilo (como 3D ou diagrama), é só me avisar!

User prompt: Crie a image agora.



User prompt: Ok, now I want a version of the article in Portuguese, so that I do not have to re-write it.

Response: Aqui está a versão do seu artigo inteiramente em português, adaptada para manter o tom natural, técnico e reflexivo que você usou no original:

Até onde acredito que podemos ir com a IA substituindo desenvolvedores de software

Eu estava originalmente tentando escrever um artigo sobre "por que acho que a IA não vai substituir os desenvolvedores de software", no entanto, acabei encontrando lacunas no meu próprio raciocínio lógico. Dito isso, ainda não acredito em uma substituição total, mas sim — como muitos já perceberam — em uma grande reestruturação do fluxo de trabalho dos desenvolvedores, sem que muita coisa mude em relação ao que se espera que eles entreguem. Abaixo estão meus pensamentos; espero que gostem.

A maior parte do meu pensamento gira em torno da responsabilidade e do equilíbrio de interesses.

É realmente verdade que podemos desenvolver software a uma velocidade extremamente alta, algo que até pouco tempo atrás mal conseguíamos imaginar. No entanto, é mais rápido gerar o código do que compreendê-lo e entender todos os seus efeitos colaterais. Mesmo que eu já tenha o código e não tenha escrito uma única linha dele, ainda serei o responsável por ele — pelo lado bom e pelo lado ruim, pelo que funciona e pelo que não funciona. O novo gargalo passou a ser entender o código e construir confiança em torno dele, e não mais escrevê-lo.

Quando usamos ferramentas de IA, atuamos como operadores dessas ferramentas e somos responsáveis pelo resultado. Sem programadores, ainda seria necessário que outra pessoa controlasse a ferramenta de IA, e essa pessoa se tornaria o novo operador. Se essa pessoa não for um desenvolvedor e for responsável por operar uma ferramenta de IA sem qualquer

conhecimento de programação, nós a chamamos de "Vibe Coder", principalmente porque ela trata o código como uma "caixa preta".

No entanto, ela ainda será a responsável pelo software. O ponto é que, como desenvolvedor de software, sou muito mais consciente de todas as consequências do meu código — se ele escala, se é legível e como se comporta.

Não ter um programador seria mais ou menos assim, do meu ponto de vista:

A mesma pessoa que hoje é um gerente não técnico descreveria qualquer implementação que desejasse. Sob essa perspectiva, você poderia dizer que o gerente é um operador, mas isso não seria diferente de um gerente operando sua equipe. Substituir programadores por um agente significaria tratar o agente da mesma forma que os desenvolvedores são tratados. Mas quem seria responsabilizado então?

Acho que é aqui que as coisas começam a se complicar. Você poderia tentar dizer que deveríamos responsabilizar a IA, mas todo mundo sabe que não funciona assim. No final das contas, sabemos que quem opera a IA é quem responde por ela. A responsabilidade poderia ser do gerente, mas aí ele seria simplesmente classificado como um "vibe coder". Ou você poderia dizer que a responsabilidade é de quem te vendeu a ferramenta de IA.

Acho que esse último cenário é o mais próximo de onde estamos agora, ou o mais perto que conseguiríamos chegar. Empresas como Anthropic, OpenAI e Google são explícitas ao dizer que não são responsáveis, mas você pode interpretar as coisas de outra forma. Um serviço ruim dessas empresas traz consequências, como perda de receita, de modo que elas são implicitamente responsáveis. A diferença é que as empresas que contratam as ferramentas de IA têm muito menos poder ao tentar direcionar o comportamento dessas provedoras.

Quando você contrata um desenvolvedor, espera adaptabilidade e um comportamento sob medida daquela pessoa. No caso de contratar uma empresa, você tecnicamente teria isso, mas seria muito mais difícil para você medir ou corrigir um comportamento indesejado.

Então, está tudo perdido? Estamos presos à estrutura organizacional atual, o que significa que a IA não mudará nada nela?

Eu não acho. Acredito que podemos conseguir muito se criarmos mecanismos para avaliar os agentes, permitindo-nos prever e medir o comportamento padrão. Deixe-me ser um pouco mais claro. Hoje, avaliamos os desenvolvedores de software primeiro em entrevistas de emprego, onde medimos o conhecimento técnico geral. Uma vez dentro de uma empresa, os desenvolvedores passam por uma série de avaliações diárias — algumas explícitas, como "o que esperamos de você nos primeiros 3 meses", e outras implícitas, definidas pelo bom senso, como:

- O código que eles escrevem é legível ou confuso demais?
- Esse colega de trabalho é fácil de gerenciar? Ou passo horas tentando explicar tarefas e revisando PRs (pull requests) cheios de bugs?
- Posso confiar que esse colega pensou em possíveis casos de borda (*edge cases*)?
- Os PRs são fáceis de revisar?

Muito disso é bem difícil de medir na prática, apesar de ser crucial.

Mas, em relação ao que *podemos* medir, como proceder?

Poderíamos definir uma linha de base (*baseline*) para o desempenho do agente. Imagine que você implementou as seguintes funcionalidades no seu sistema no passado:

- Login
- Cadastro (Sign up)
- Encerramento de conta
- Reabertura de conta

Você pode não se importar com cada linha de código, mas se importa com o comportamento da sua aplicação e com a escalabilidade dela. Você poderia, por exemplo, tirar um retrato (*snapshot*) da sua aplicação antes de essas funcionalidades serem implementadas, pedir para o agente implementá-las e verificar se a implementação atende ou não aos testes de integração que você já tinha para aquelas funcionalidades. O objetivo aqui é medir o quão bem o agente consegue implementar recursos de uma forma que seja aceitável para o seu sistema.

Quanto mais funcionalidades seu sistema tiver, mais testes você poderá criar.

Outros aspectos também podem ser medidos. Você poderia implementar funções de aptidão (*fitness functions*) para ajudar a criar testes para coisas como:

- Desempenho
- Se a implementação respeita a arquitetura do sistema

Acredito que deveríamos permitir e esperar que os agentes nos façam perguntas para produzir melhores implementações, assim como permitimos que os desenvolvedores façam perguntas às pessoas de negócios (*stakeholders*).

Isso é muito parecido com os desenvolvedores de software criando seus próprios benchmarks para avaliar agentes, desenvolvedores ou equipes. Os agentes podem ser aprimorados e avaliados regularmente em relação a esses benchmarks. Eles podem ser modificados e reavaliados; podemos ter benchmarks diferentes para papéis diferentes (ex: Frontend, Backend, SRE) e podemos até avaliar o custo por tarefa.

Voltando ao meu argumento anterior sobre responsabilidade: podemos agora responsabilizar os agentes? Não, ainda não podemos. Os responsáveis seriam:

1. Quem cria os benchmarks
2. Quem aprova o código para ir para produção
3. Quem cria o agente

Esses três papéis podem ser desempenhados pela mesma pessoa, mas não precisam ser, e a responsabilidade final cabe ao nº 2.

No entanto, uma empresa poderia criar um benchmark e contratar um agente que tenha um desempenho melhor do que qualquer agente que ela fosse capaz de criar internamente. Você poderia ter alguém vendendo um agente especializado para fintechs que atua em um nível

específico em benchmarks específicos de fintechs, por exemplo. Sinceramente, acredito que apenas o item nº 3 poderia vir de fora da empresa.

Então, sim, acredito que isso é o mais próximo que poderíamos chegar de agentes substituindo desenvolvedores de software: uma estrutura de equipe na qual testes padronizados são aplicados aos agentes para avaliar o quão bem eles criam especificações e funcionalidades que atendam às expectativas, onde uma empresa poderia até contratar agentes externos simplesmente porque eles performam melhor.

Alguns pensamentos finais:

Por que isso seria tão difícil?

1. Todo mundo quer uma solução pronta para o uso (*plug-and-play*), e ninguém quer o trabalho e o custo de criar benchmarks.

2. É muito difícil testar conceitos subjetivos como "bom design" e "boa arquitetura". Pelo menos, eu não conheço uma maneira clara, determinística e matemática de avaliar se o código segue os princípios do SOLID ou se atende aos preceitos da Clean Architecture (Arquitetura Limpa).

Esses são os meus pensamentos; espero que tenham gostado! :)

User prompt: Há alguma solução no mercado ou open source que vá na direção do que eu descrevi no meu artigo?

Response: Sim, a indústria e a comunidade *open-source* estão caminhando exatamente na direção do seu raciocínio. A transição de "avaliar código por legibilidade" para "avaliar comportamento via testes e *fitness functions*" já é a principal fronteira da engenharia de software baseada em IA.

Aqui estão as principais soluções de mercado e *open-source* que validam e já implementam o que você descreveu no seu artigo:

1. Benchmarks de Agentes (A sua ideia de "Snapshots" e Testes de Integração)

O seu conceito de tirar um retrato do sistema, pedir para o agente implementar a *feature* e rodar testes de integração contra isso é, hoje, o padrão-ouro na pesquisa de IA.

- **SWE-bench (e SWE-Bench Pro):** É o *framework open-source* mais famoso atualmente para avaliar sistemas de codificação de IA. Ele faz exatamente o que você propôs: fornece ao agente uma *issue* real do GitHub e um retrato (*snapshot*) de um repositório real. O agente precisa gerar um *patch* de código, e a avaliação é feita rodando a suíte de testes do próprio repositório para verificar se o problema foi resolvido e se nada foi quebrado no processo.

- **ProgramBench e Terminal-Bench:** Enquanto o SWE-bench foca em *features* e *bugs*, o ProgramBench avalia se os agentes conseguem reconstruir programas do zero usando milhares de testes comportamentais. Já o Terminal-Bench avalia os agentes em ambientes

reais de terminal (compilando código, instalando dependências e rodando testes), garantindo que eles funcionem na prática e não apenas gerem "código bonito".

2. Funções de Aptidão e Arquitetura (Sua ideia de testar "Bom Design" e SOLID)

No final do artigo, você mencionou a dificuldade de testar determinística e matematicamente se um código segue arquitetura limpa ou princípios SOLID. O mercado está atacando isso de duas formas:

- **ArchUnit (e ArchUnitTS):** São bibliotecas tradicionais que permitem escrever *fitness functions* determinísticas no código (ex: garantir que a camada de "Domain" não acesse o "Banco de Dados"). Elas automatizam as decisões de arquitetura em cada *commit*.
- **LLMs como "Fuzzy Fitness Functions" (LLM-as-a-Judge):** Como conceitos de "coesão" e "design" são interpretativos, desenvolvedores estão criando *fitness functions* que usam LLMs com *outputs* estruturados nas *pipelines* de CI/CD para atuarem como "juízes". Eles recebem as regras da arquitetura da empresa no *prompt* e rodam como testes automatizados em cada *pull request*.
- **CodeEvolve:** É um *framework open-source* de descoberta algorítmica que utiliza algoritmos evolutivos combinados com LLMs. Ele implementa explicitamente *fitness functions* para medir de forma transparente e rigorosa a qualidade técnica das soluções geradas pelos agentes.

3. Plataformas de Governança e Avaliação Contínua (O papel do "Operador")

Para lidar com a gestão e a responsabilização dos agentes — evitando o cenário do *"vibe coder"* às cegas —, o mercado está criando plataformas focadas em testar e monitorar esses agentes corporativos:

- **Google Antigravity:** É uma plataforma de desenvolvimento agentivo que permite construir, testar e fazer o *deploy* de *workflows* de IA autônomos para engenharia de software e operações de sistema, oferecendo *pipelines* determinísticas e etapas de verificação formal para gerenciar os agentes.
- **LangGraph + LangSmith / OpenAI Agents SDK:** Frameworks como o LangGraph (geralmente acoplado ao avaliador LangSmith) permitem que as equipes construam arquiteturas e rastreiem toda a execução do agente. Em vez de testar apenas o código final, eles testam as camadas de roteamento, o acionamento de ferramentas (*tool calling*) e o estado da memória do agente para prever falhas em escala.

O seu artigo está cirurgicamente alinhado com o estado da arte da área. A dor que você identificou — a falta de testes padronizados e métricas claras para "contratar" e responsabilizar agentes — é exatamente o problema que essas ferramentas estão tentando resolver hoje.

User prompt: Mas sobre a minha ideia de benchmarks, o principal ponto é que as empresas criem os seus próprios benchmarks. Há ferramentas que auxiliem as empresas a criarem os próprios benchmarks para os agentes ou skills que utilizam em seu sistema?

Response: Sim, o mercado percebeu exatamente essa necessidade. Avaliar um agente em um repositório público não garante que ele entenda as regras de negócios, a arquitetura interna ou as bibliotecas proprietárias de uma empresa específica.

Para resolver o problema do "overhead de criar benchmarks" que você mencionou no seu artigo, surgiram ferramentas e plataformas focadas na criação de **Evals (Evaluations)** personalizados. Elas ajudam as empresas a construírem suas próprias suítes de testes para agentes de IA:

1. Ferramentas para transformar repositórios privados em Benchmarks de Código

- **SWE-bench Toolkit (Open Source):** Embora o SWE-bench seja famoso pelo seu *leaderboard* público, a equipe por trás dele disponibilizou as ferramentas de infraestrutura (`swe-bench-docker` e os *scripts* de coleta). Isso permite que uma empresa aponte a ferramenta para os seus próprios repositórios privados no GitHub/GitLab, converta *Pull Requests* do passado em cenários de teste isolados e avalie se o agente consegue reproduzir a solução que o desenvolvedor humano fez. É a execução literal da sua ideia de usar "snapshots" passados do sistema.

2. Plataformas de LLMOps para Benchmarks Comportamentais e de Skills Se o agente não escreve apenas código, mas também executa ações no sistema (fechar contas, analisar logs, etc.), as empresas usam plataformas de *Tracing* e *Evaluation*:

- **LangSmith (da LangChain):** É uma das ferramentas mais utilizadas para isso hoje. A empresa pode criar "Datasets" que servem como benchmarks customizados. Você define a entrada (o que foi pedido ao agente), o estado inicial do sistema e os "Evaluators" (que podem ser *scripts* em Python, testes de integração ou *LLMs-as-a-judge*). Toda vez que você modifica o agente, a plataforma roda esse seu benchmark customizado para garantir que as *skills* não regrediram.

- **Braintrust e Phoenix (Arize AI):** São plataformas corporativas inteiramente focadas em testes de regressão para IA. Elas permitem que você colete logs de interações reais do seu sistema, classifique o que é considerado "comportamento aceitável" e transforme isso em um benchmark automatizado que roda sempre que um novo agente ou versão é testado.

3. Ferramentas para Testes de Integração em CI/CD (O formato "Plug and Play")

- **Promptfoo (Open Source):** É uma ferramenta de linha de comando (CLI) muito popular entre desenvolvedores para testar agentes e prompts de forma determinística. Você cria arquivos de configuração locais onde define as *skills* do agente e cria "Assertions" (afirmações). Por exemplo, você pode configurar o Promptfoo para disparar o agente, deixá-lo tentar implementar a *feature* e, no final do teste, rodar um *script* local que checa se o seu banco de dados ou sua API retornou o status correto, simulando o teste de integração que você descreveu.

- **AgentOps:** Uma plataforma emergente construída especificamente para monitorar e avaliar agentes autônomos (como aqueles construídos em CrewAI ou AutoGPT). Ela ajuda a empresa a criar benchmarks de custo (quanto o agente gastou de API para resolver a *issue*), tempo e taxa de sucesso em *skills* específicas dentro do ambiente privado da empresa.

O desafio atual dessas ferramentas é justamente o que você concluiu: convencer as empresas de que o esforço inicial de configurar esses benchmarks customizados — estruturando os *datasets* internos e conectando os testes de integração aos *frameworks* de IA — vale a pena a longo prazo.

User prompt: Sempre responda em 100 palavras ou menos. Ok, agora me ajude a encontrar falhas na minha teoria, na minha linha de raciocínio e na construção de benchmarks.

Response: Aqui estão potenciais falhas na sua teoria:

1. **Paradoxo do Esforço:** Se times já evitam criar testes simples, o custo e tempo para manter *benchmarks* complexos para agentes inviabiliza a adoção em larga escala.

2. **Armadilha do *Snapshot*:** Avaliar agentes contra testes do passado força a IA a imitar o legado. A IA pode gerar uma solução superior que falhará simplesmente por não seguir a implementação original engessada.

3. **Transferência de Complexidade:** O gargalo cognitivo apenas muda de lugar. Criar *fitness functions* arquiteturais perfeitas costuma ser muito mais difícil e abstrato do que programar a própria *feature*.

User prompt: Teremos um debate aqui. Quero que você sempre contra argumente. 1 - Não criar testes sempre foi visto como má prática pois essencialmente o seu código será testado, ou de maneira automatizada, ou pelos seus clientes. É melhor que a empresa tenha testes. Realmente, esse cenário não é viável para uma empresa que nem pensa em focar nos testes básicos, mas quem peca é a empresa em não fazer o mínimo. 2- No que diz respeito a 'regras de negócio', dificilmente há uma 'solução superior'. As regras de negócio dificilmente são melhores ou piores. Elas ou seguem o padrão ou não, o espectro é muito menor do que 'uma solução melhor ou pior'. 3- Sempre haverá um gargálo, pois sempre que você otimizar em uma etapa absurdamente, outra ficará para trás. Sempre haverá espaço para melhorias, o que vai mudar é onde. Além disso não disse que as funções arquiteturais tem que ser perfeitas, elas por si só são passíveis de evolução.

Response:

1. **Velocidade vs. Perfeição:** Startups sobrevivem lançando rápido. Exigir *benchmarks* complexos e rigorosos para IA destrói o *time-to-market*. O mercado recompensa velocidade de entrega, não a perfeição da cobertura de testes.

2. **Engessamento:** A IA pode resolver um problema de negócios criando um fluxo ou algoritmo muito mais eficiente, mas será reprovada imediatamente pelo seu teste rígido, que foi desenhado para validar a lógica antiga e legada.

3. **Custo ilusório:** Se o novo gargalo for "escrever e evoluir *fitness functions*", você não automatizou o custo. Apenas trocou o desenvolvedor por um "engenheiro de testes" igualmente caro.

User prompt: 1- Isso que eu falei não é algo pensado em empresas que estão na luta pela sobrevivência, ou correndo atrás do relógio, é para empresas que tem fôlego para evoluir arquiteturalmente. Nem sempre 'agora' é o momento ideal para uma reestruturação pesada dos processos. 2- Simples, não devemos testar implementação, mas sim comportamento. No exemplo dado, eu não faço nenhuma questão que uma IA escolha um algoritmo específico, mas que o resultado tenha uma performance mínima. 3- Essa troca não é 'um por um' e você não tem como

argumentar que o preço se mantém. O preço poderia até mesmo ser maior. Assumindo que a empresa vá fazer o uso da IA e de agentes, ela vai querer profissionalizar esse processo. O que eu falei é o processo de maturação, não fazer a troca que você falou é simplesmente: 'vamos manter as coisas sem padrão e sem regras porque mudar seria mais caro'. Tudo isso tentamos fazer com o objetivo de deixar processos mais ágeis e eficientes.

Response:

1. **Fôlego corporativo é ilusão:** Até empresas gigantes exigem ROI rápido. Se o *benchmark* não provar valor imediato, a diretoria corta o orçamento. Maturidade não significa cheque em branco.

2. **Comportamento mascara falhas:** Testar apenas *output* é perigoso. A IA pode atingir o comportamento esperado usando **gambiarras ou introduzindo vulnerabilidades** silenciosas que testes comportamentais de caixa-preta jamais detectarão.

3. **Burocracia disfarçada de agilidade:** Profissionalizar pode gerar um **novo gargalo engessado**. A empresa corre o risco de gastar mais tempo dando manutenção nas regras e validando agentes do que usufruindo da velocidade que a IA prometeu entregar.
